Project status summary

Global Flood Event Data Pipeline — generated 2026-04-27

Working (validated end-to-end against Supabase)

1. Database schema

- `db/schema.sql` — Idempotent DDL: PostGIS extension, three schemas (`raw` / `staging` / `marts`), 5 raw JSONB tables, `raw.ingestion_log` audit table, canonical `staging.flood_events` with H3 + PostGIS, plus forward-migration `ALTER TABLE … ADD COLUMN IF NOT EXISTS` statements that bring any pre-existing tables up to date.

2. Configuration & shared DB layer

- `config/settings.py` — Loads `.env`, builds `DATABASE_URL`, exposes `H3_RESOLUTION` and other tunables.
- `db/client.py` — Singleton SQLAlchemy engine tuned for the Supabase pooler (TCP keepalives, `pool_pre_ping`, `pool_recycle=300`), `apply_schema_sql()`, `insert_raw_records()`, `truncate_raw_table()`, `log_ingestion()`, `execute_with_retry()` with exponential backoff on `OperationalError`, and `_clean_for_json()` for NaN-safe payloads.

3. Ingestion (5 sources, 14,708 rows total)

- `ingestion/common.py` — `download_to()` with SHA-256 + sidecar `.meta.json`, plus `parse_with_fallback()` for parser-time recovery.
- `ingestion/ingest_dartmouth.py` — **913 rows** (Dartmouth Flood Observatory).
- `ingestion/ingest_glofas.py` — **5,503 rows** (Global Active Archive of Large Floods CSV).
- `ingestion/ingest_copernicus_ems.py` — **335 rows** (filtered to floods only).
- `ingestion/ingest_emdat.py` — **6,178 rows** from bundled seed.
- `ingestion/ingest_reliefweb.py` — **1,779 rows** via the public REST API.

4. Unified canonical model

- `transformations/transform.py` — Per-source `_normalize_*` functions, H3 v3 indexing, PostGIS `ST_MakePoint`, upsert on `(source, source_event_id)`. All 14,708 rows land in `staging.flood_events`.

5. Marts (6 views)

- `transformations/marts.py` — `marts.flood_events`, `flood_events_by_region`, `flood_events_by_h3`, `flood_frequency_by_basin`, `flood_events_by_month`.

6. Data quality

- `validation/data_quality.py` — 7 row-level checks plus a per-source rollup.
- `data/logs/data_quality_report.md` — All 7 checks pass (`OK`) against the loaded data.

7. Orchestration

- `dags/flood_event_pipeline_dag.py` — Apply schema → 5 ingestions in parallel → transform → marts → DQ.

8. REST API

- `api/main.py` — FastAPI: `/health`, `/flood-events`, `/flood-events/by-region`, `/by-time`, `/by-severity`, `/by-h3`, `/analytics/frequency-by-basin`. Returns valid JSON (verified live).
- `api/Dockerfile` — Standalone container image.

9. Container orchestration

- `docker-compose.yml` — Airflow with `_PIP_ADDITIONAL_REQUIREMENTS`, individual mounts for each source package (`config/`, `db/`, `ingestion/`, `transformations/`, `validation/`), plus a separate `api` service that only mounts `config/` and `db/`.

10. dbt scaffolding (optional)

- `dbt/dbt_project.yml`, `dbt/profiles.yml`
- `dbt/models/sources.yml` — declares the 6 sources.
- `dbt/models/staging/stg_dfo.sql` — example staging model.
- `dbt/models/marts/flood_events.sql` — passthrough mart.

11. Documentation

- `README.md` — Architecture diagram, layout, config, run instructions, API examples, known limitations, next steps.
- `docs/data_sources.md` — Per-source URLs, formats, license / access notes, fallback behaviour.

Problems faced (and how they were fixed)

#	Problem	Root cause	Fix
1	`schema.sql` wasn't valid SQL — it was just a column listing without `CREATE TABLE`	Original file was a sketch	Rewrote `db/schema.sql` as full idempotent DDL
2	`staging.flood_events` already existed in Supabase from a prior run, missing the new columns (`affected`, `flood_impact_index`, `loaded_at`)	`CREATE TABLE IF NOT EXISTS` silently no-ops when the table is already there	Added `ALTER TABLE … ADD COLUMN IF NOT EXISTS` migrations + a DO-block for the unique constraint

3	Dartmouth XLSX URL returned **HTTP 200 with HTML**; `download_to` declared "success" but the parser blew up with `BadZipFile`	A successful HTTP status doesn't guarantee parsable bytes	Added `parse_with_fallback()` in `ingestion/common.py` — retries with the seed file when the parser raises
4	Supabase pooler killed `INSERT` mid-batch (`server closed the connection unexpectedly`) on big GloFAS / EM-DAT loads	pgBouncer idle-kill plus large multi-row VALUES blocks	Engine-level tuning in `db/client.py`: chunk = 50, TCP keepalives, `pool_pre_ping`, `pool_recycle=300`, plus `execute_with_retry()` with exponential backoff and `_reset_engine()` on failure
5	`psycopg2.errors.InvalidTextRepresentation: Token "NaN" is invalid` on EM-DAT JSON insert	Pandas serialises empty cells as float `NaN`, which Postgres JSONB rejects (RFC 8259 strict)	Added recursive `_clean_for_json()` plus `json.dumps(..., allow_nan=False)` in `db/client.py`
6	DQ check showed 1,783 rows with severity > 10	EM-DAT's "Magnitude" column for floods is **inundated area in km²**, not a normalized severity	Moved EM-DAT `Magnitude` → `flood_impact_index` and set `severity = NULL` for that source in `transformations/transform.py`
7	`TypeError: Invalid argument(s) 'executemany_values_page_size'` to `create_engine()`	Wrong argument name for SQLAlchemy 2.0	Switched to `insertmanyvalues_page_size=10`
8	`geopandas` / `shapely` in requirements would force GDAL / GEOS into the airflow image	Wheels exist but pull heavy native deps	Removed both from `requirements/base.txt` — never imported anyway, since PostGIS handles geometry server-side
9	Transient DNS failure mid-EM-DAT during testing (`could not translate host name`)	Local network blip	Already covered by `execute_with_retry`; just re-ran the one source

---

What's still missing / known limitations

Data sources

- **GloFAS reanalysis grids** — currently uses the public Active Archive CSV only. The Copernicus CDS API branch is a placeholder. Set `CDS_API_URL` and `CDS_API_KEY` and add a `cdsapi` call in `ingestion/ingest_glofas.py`.
- **EM-DAT** — requires a free login; bulk download is gated. The pipeline only reads the seed CSV unless you set `EMDAT_DOWNLOAD_URL` to a session-signed URL.
- **Copernicus EMS** — no stable public JSON feed. Falls back to the bundled CSV unless `COPERNICUS_EMS_FEED_URL` is provided.

Schema gaps

- **River basin** — `staging.flood_events` has no `basin` column. `marts.flood_frequency_by_basin` currently approximates basin by country. EM-DAT preserves "River Basin" in the raw JSONB, so promoting it is a one-line schema change plus a view update.
- **Polygon geometries** — only point geometries are stored. Polygon shapefiles from Copernicus EMS or DFO would need a `geometry_polygon GEOMETRY(MultiPolygon, 4326)` column and updated centroid-for-H3 logic.

Coverage gaps

- **Per-source coordinate availability is uneven**: Dartmouth has 100% lat / lon, EM-DAT has ~16%, GloFAS / Copernicus EMS / ReliefWeb seeds have 0%. This is a data-source reality, not a bug.
- **Severity** is only populated for Dartmouth and GloFAS (1–2 scale). EM-DAT and ReliefWeb don't expose one.

Pipeline / infrastructure

- **dbt isn't wired into the Airflow DAG** — `dbt-core` would need to be added to `_PIP_ADDITIONAL_REQUIREMENTS` and a `BashOperator` task added for `dbt run`. Models exist standalone in `dbt/`.
- **`SequentialExecutor`** is fine for now but only runs one task at a time. Switching to `LocalExecutor` requires a real metadata DB (Postgres container) instead of the bundled SQLite.
- **No automated test suite** — no `pytest` tests yet. Tests for `_normalize_*` functions, `_clean_for_json`, `parse_with_fallback`, plus one integration test against a Postgres test container would be ideal.
- **No Great Expectations / Soda** alongside `data_quality.py` — current checks are inline SQL only, not a versioned suite.

Repository hygiene

- The codebase has been restructured into logically separated top-level packages: `config/`, `dags/`, `db/`, `ingestion/`, `transformations/`, `validation/`. The old `scripts/` folder now only contains the one-off `_make_status_pdf.py` utility. Empty placeholders that previously lived in those folders have been removed.

Docker / runtime

- **First-run cost** — `_PIP_ADDITIONAL_REQUIREMENTS` reinstalls dependencies every time the airflow container starts. For production, build a custom Airflow image instead.
- **`.env` is bind-mounted** into the airflow container at startup; ensure it isn't committed.

Security

- **Default Airflow credentials** are `admin / admin` (intentional for local). Change before any non-local deployment.
- **No CORS restrictions** in `api/main.py` — currently `allow_origins=["*"]`. Lock down before exposing.

Recommended priority for next session

1. **Promote `river_basin` to a real column** in `staging.flood_events` so the basin mart becomes hydrologically accurate (1 schema change + 1 transform line + 1 view).
2. **Add a small `tests/` folder** with pytest cases for each `_normalize_*` function and `_clean_for_json` — gives regression safety as data shapes drift.
3. **Wire dbt into the DAG** as a final task — turns the dbt models from documentation into actual run artifacts.
4. **Build a custom Airflow image** so dependencies don't reinstall on every container start.